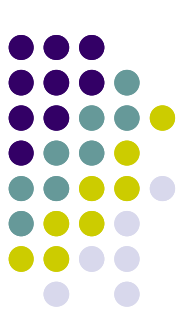
Intro Design Patterns



Architecture of Computer Games

SE 456

Ed Keenan

4 January 2023

13.0.11.3.17 Mayan Long Count

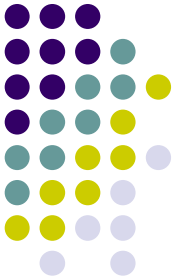
Overview

- Design Patterns
  - Intro
  - Need
  - Cohesion / Coupling
  - Benefits
  - Performance
- Patterns
  - Singleton
  - Factory

Intro to Design Patterns

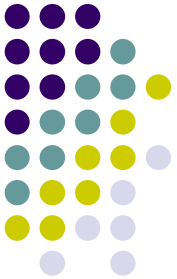
- Design Patterns and Object Oriented
  - Intermixed
  - Core Object Oriented principles steer Patterns
- Reference and Links
  - http://www.uml-diagrams.org/class-reference.html
  - https://www.dofactory.com/net/design-patterns
  - http://www.oodesign.com/factory-pattern.html

Design Patterns - quick



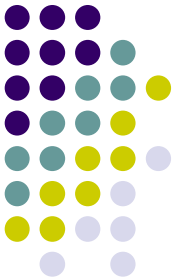
- Design pattern is a general reusable solution to a *commonly occurring problem* within a given context in software design.
  - a description or template for how to solve a problem
- Patterns are formalized best practices.
- Show relationships and interactions between classes or objects

Need for Object-Oriented



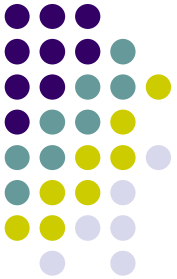
- *Functional Decomposition* is a natural way to deal with complexity.
  - Algorithmic decomposition breaks a process down into well-defined steps
  - Example: appending a string to a file
- Natural side effect
  - Saddles the program with too much responsibility.
  - Does not allow for easy, low risk changes
    - Many bugs originate with changes to code

Object-Oriented Paradigm



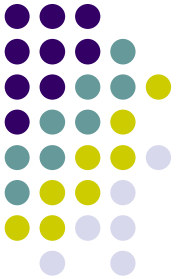
- Requirements will always change
  - As you understand the problem
    - you will change the requirements
  - Change, change, change!!!
    - Will never stop
- Benefits of Object Orientation
  - Can *contain areas of change*
  - Insulate code from the effects of change more easily
- Design code so that the impact of changing requirements is much less dramatic.

Cohesion / Coupling



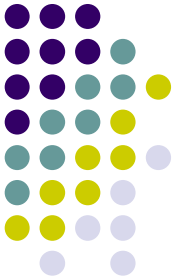
- **Cohesion**
  - How closely the operations in a routine are related
- **Coupling**
  - How strongly a routine is related to other routines
- GOAL
  - Create routines with internal integrity
    - (strong cohesion)
  - Create routines with small, direct, visible, and flexible relations to other routines
    - (loose coupling)

Design Patterns



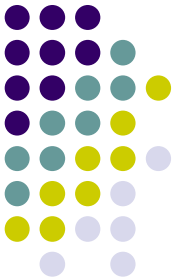
- Design Patterns and Object-Oriented design reinforce each other.
- Pattern
  - Solution to a problem in a context
  - Describes *a core way* to solve a problem, such that you can use it over and over again without ever doing it the same way twice
- Patterns exits for almost any design problem
  - May be combined to solve complex problems

Gang of Four



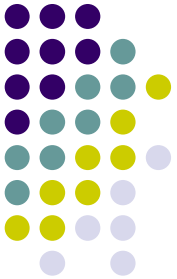
- ***Design Patterns: Elements of Resusable Object-Oriented Software***
  - Gamma, Helm, Johnson, Vlissides
- Applied the idea of patterns to software design
  - Calling them design patterns

Design Patterns Benefits



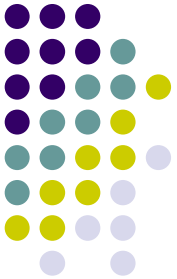
- Help with reuse and communication
  - Give a higher perspective on analysis and design
- Helps use see the forest and the trees
  - Ability to be external and away from the details
- Improve
  - Communication of Code
  - Modifiability of Code
  - Maintainability of Code

Design Patterns Benefits



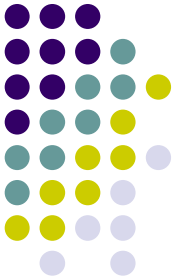
- Illustrate basic Object-oriented principles
  - Design to interfaces
  - Favor Aggregation over inheritance
  - Find what varies and encapsulate it
- Offers alternatives to large inheritance hierarchies

OO vs Performance



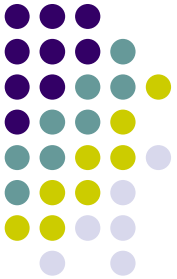
- Object-oriented design is sometimes at odds with performance
  - Design standpoint
    - it's best to encapsulate everything
    - treat those objects uniformly
    - reuse them
    - extend them
  - Performance standpoint - we want lean and mean
- ***Thunder Dome***
  - Two man enter one man leaves

Inheritance issues



- Understand the difference between class inheritance and interface inheritance (or subtyping).
  - ***Class inheritance*** defines an object's implementation in terms of another object's implementation.
    - mechanism for code and representation sharing.
  - ***Interface inheritance*** (or subtyping) describes when an object can be used in place of another.
- I hate those names!

Inheritance issues



Here's the problem they are both inheritance.... so what names should they have?

A) LinkList is a base class...

- is implementing the derived class in terms of the base class

B) Dog is base class and Poodle is derived is inheritance...

- "is-A" or polymorphic.

Gang of Four

A is call Class Inheritance

B is called Interface Inheritance

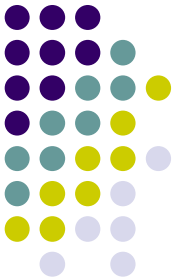
CONFUSING...

A is a Mix-In of only one class (LinkList)

B is a Polymorphic class... the Derived is-A Base class.

From now on.. I'm using Mix-In, and Polymorphic class as my description.

Mixin



- Mixins
  - language concept that allows a programmer to inject some code into a class
  - a style of software development, in which units of functionality are created in a class and then mixed in with other classes
- Mixin class acts as the parent class
  - containing the desired functionality.
  - Subclass can then inherit or simply reuse this functionality
    - but not as a means of specialization
- Mixin will export the desired functionality to a child class, without creating a rigid, single "is a" relationship.
  - Important difference between the concepts of mixins and inheritance,
    - child class can still inherit all the features of the parent class, but, the semantics about the child "being a kind of" the parent need not be necessarily applied

Thank You!

- Questions?